Closebook: Product Requirements for TestSprite

Test URL: the Cloudflare tunnel URL Hermes prints (`make testsprite-url`). Start at `/app`.

PIN for every approval: the instance's own PIN, in `.env` as `TESTSPRITE_PIN`. It is deliberately NOT written here — this file is in a public repository, and a credential that is committed is a credential that has to be rotated.

Login: none.

Environment: Airwallex sandbox only. No real money moves.

> **Merged document.** The original TestSprite PRD's structure, screens and flows are kept as they were. What changed is the expectations section, corrected to what the app actually does, plus a new section 6 on testing in a shared workspace.

---

1. What Closebook is

Closebook is a real-time bookkeeping agent for a small business that gets paid by customers and pays suppliers in several currencies through Airwallex.

- A customer payment lands in the Airwallex account.
- Closebook matches it to the right invoice. Rules decide first, and an AI model only proposes when the rules can't decide.
- Anything unclear waits for a person, who approves with a PIN.
- When enough cash has landed, Closebook pays the suppliers that were waiting on it. It never takes cash below the reserve floor, and anything above the automatic limit needs a PIN.
- It follows each transfer until it's paid, failed or returned, and keeps a balanced ledger.

Core principle: the model proposes, code decides. The AI's confidence is capped at 0.89, below the 0.90 auto-apply band, so the model can never move money on its own.

---

2. Screens (left sidebar on desktop, bottom tab bar on mobile)

Tab	Purpose
Home	Cash on hand, promised to suppliers, in flight, free to spend (with its working shown as a waterfall), suspense, refunds owed, today's checklist, problems, books balance
Money in	One timeline of every deposit and how each was matched, each row expanding into its receipt and its resolution; the Simulate deposit form (inside the Sandbox controls panel)
Money out	Bills (payables), their plan status, transfers, and the **Sandbox controls** panel holding Settle / Fail / Return
Needs your OK	Proposals and payments waiting for a PIN; suspense items to resolve
AI activity	Each model call: what it proposed, its claimed confidence, the code's verdict

Setup	Settings, Customers, Invoices, Suppliers, Bills, Start fresh

3. Key rules (what tests should assert)

- **Free to spend** = cash − promised − in flight − unattributed suspense − refunds owed − reserve floor. Home shows this working.
- **Suspense (unmatched money) is shown but never spendable** until it's resolved.
- **Suspense resolves in exactly one of three ways:** Match to an invoice, **Refund owed** (needs a reason, stays non-spendable), or **Release as income** (needs a note, becomes spendable).
- **Every money decision needs the PIN.** A wrong PIN is refused with a clear message and changes nothing.
- **Any action done twice** (approve, resolve, settle) is refused the second time and writes nothing.
- **An invoice can never be paid beyond its open balance.**
- **A payment above the automatic limit, or one that would take cash below the floor, is held with a plain-English reason** — and a payment over the limit that is *otherwise payable produces an approval card in Needs your OK*. It does not sit in a Held list that no person can release.
- **A PARTIAL PAYMENT BELOW 0.90 IS HELD FOR APPROVAL.** It is proposed, never applied on the strength of an amount that merely looks close, and it appears in Needs your OK with the invoice it claims and its confidence.
- **A FAILED OR RETURNED TRANSFER ENDS CANCELLED, WITH ITS REASON SHOWN NEXT TO IT.** It posts nothing to the ledger. The bank reports a return as CANCELLED, which on its own reads as "someone stopped this", so the reason the bank gave must be shown.
- **A DEPENDENCY HOLD IS NOT A RESERVE HOLD.** They read similarly and need opposite responses: a dependency hold clears itself when the customer pays and **no approval can release it**, while a reserve hold clears the moment a person approves it or cash arrives. A part-payment does **not** release a dependent payable — the dependency is met only by a fully paid receivable.
- **A bill held for more than one reason states every reason**, not just the first, because two holds need two different actions.
- **"The books balance: yes" must stay true after every action.**
- **The payment tracker marks only FAILED, RETURNED and CANCELLED with a red X**, at the step where it stopped. A healthy in-flight transfer never shows one, however many earlier attempts on that bill failed.
- **No page shows a raw error, stack trace or HTTP 500.** Every button ends in a plain sentence within a few seconds. Vendor errors are replaced by a sentence, with the machine's own words behind a **Detail** toggle.
- **The Sandbox controls live in one collapsed, dashed-border panel labelled "sandbox only"**, holding the deposit simulator and Settle / Fail / Return. Fail and Return ask for confirmation, each row names its payment, and **the normal screens show none of it** — no sandbox control appears on Home, AI activity or Setup.

4. Test flows (run in this order on an empty instance)

A. Landing and navigation

- Open `/`. The landing page shows **"Every deposit matched. Every payment checked."** and a **Launch app** link in the top nav.
- Launch app goes to `/app` Home. Every tab opens without errors. Back to site returns to `/`.
- The landing page carries **one** live proof, counted from the app's own tables, and invents no evidence: no testimonial, no decorative figures.

B. Setup

- Settings: budget currency USD, reserve floor 5,000, automatic limit 2,500. Save, reload, and the values persist.
- Customers: add "Brightline Retail" (NG) and "Harrow & Co" (GB).
- Invoices: INV-1001 to Brightline, USD 12,000. INV-1002 to Harrow, GBP 8,000.
- Suppliers: add a USD supplier (US, LOCAL). The app creates a real Airwallex beneficiary and shows its ID. Invalid bank details must show a clear validation message, not a crash.
- Bills: PO-2001 to that supplier, USD 1,800, dependent on INV-1001.
- Duplicate invoice or bill numbers are refused with a message.

C. Money in, exact match

- On Money in, simulate a USD 12,000 deposit with reference "INV-1001".
- It matches INV-1001 as EXACT and auto-applies. INV-1001 shows paid and cash rises by 12,000.

D. Partial payment

- Simulate GBP 4,800 with reference "INV-1002 part".
- INV-1002 shows 4,800 paid, GBP 3,200 open. Any chase step shows GBP 3,200, not USD.

E. Unclear match goes to approval

- Add invoice INV-1003 for a new customer "Lumen Stores", USD 4,400.
- Simulate USD 4,400 from "Apex Factors Ltd", reference "for the lumen stores account".
- It appears in Needs your OK as PROPOSED, with confidence at most 0.89.
- The **Model said / Code decided** receipt shows the model's claimed value **struck through** beside the code's final value, the rule that fired, and the verdict. The model's claimed confidence is never reported as the final one.
- Approve with the wrong PIN: refused, nothing changes.
- Approve with the correct PIN: INV-1003 is paid, suspense drops, and a success sentence appears.
- Approve again: refused, "already resolved".

F. Over-application guard

- Simulate a second USD 4,400 with the same Lumen reference.
- Match to INV-1003 is no longer offered (or is refused). The app explains that the invoice is already paid.
- Resolve as **Refund owed** with reason "duplicate payment". Refunds owed shows 4,400 and free to spend does not change. **The row's state reads "Refund owed", not PROPOSED** — the badge follows the resolution, not the stale matcher verdict.

G. Unknown payer

- Simulate USD 7,000 from "Unknown Party", reference "ZXQ-99881".

- It lands in suspense. Cash rises, free to spend does not.
- **Ask the model again** works without a PIN and posts no ledger entries.
- Resolve as **Release as income** with a note. Suspense drops and free to spend rises by 7,000.

H. Paying suppliers

- After INV-1001 lands, PO-2001 (USD 1,800, under the limit) is sent automatically.
- Money out shows a transfer ID and PROCESSING, with **no red X** on its tracker.
- **Add a bill above 2,500.** It appears **in Needs your OK as an approval card** with a reason, and is sent only after a PIN approval.
- A bill that would break the reserve floor is held with a sentence naming the shortfall, and stays under **Held — and not waiting on you**, because no approval can release it.
- A bill held for two reasons at once states both.

I. Transfer outcomes (Sandbox controls on Money out)

Open the collapsed **Sandbox controls** panel first; the controls are not on the page otherwise. Each row names its payment (for example "PO-7001 · Atlanta Packaging · USD 1,800.00").

- **Settle** a PROCESSING transfer. It shows PAID, cash drops by the amount, free to spend is unchanged, and the bill leaves the held/plan lists.
- Press Settle again: "already settled", no new ledger rows.
- **Fail** another transfer (confirm when asked). It ends CANCELLED **with its failure reason shown next to the status**, its tracker shows a red X at the step it stopped at, and nothing posts to the ledger. Problems offers "Re-plan and ask for my OK".
- **Return** another one (confirm when asked). Same as Fail, **with a "returned" reason shown next to CANCELLED**.

J. Integrity checks (run after every flow)

- Home shows "The books balance: yes".
- A bill never shows as both paid and on hold.
- Free to spend equals the working written under it — the waterfall segments and the closing figure sum to the tile.
- No page contains a raw stack trace or a vendor error.

K. Mobile

At 390px wide: no horizontal scrolling on any tab, tiles at full width, bottom tab bar visible, and forms usable. The waterfall stacks its segments into labelled rows rather than a proportional bar.

---

5. Out of scope

- Real money and production Airwallex.
- **Card issuing.** Beat 4 shows a policy verdict only, not a real card. That's expected, not a bug.
- **Refund transfers.** "Refund owed" records the liability only.
- **Start fresh:** don't test it on a shared run, since it archives the workspace. Reset with `make testsprite-reset` instead.

6. Testing in a shared workspace

Several runs share one sandbox account and one Airwallex key, and two instances of the app can see each other's deposits. Follow these so one run does not corrupt another's evidence.

- **Give every record a unique suffix.** Names, invoice numbers, bill numbers and references must be unique per run — `QA-OVERLIMIT-7f31`, not `QA-OVERLIMIT-001`. Two runs using the same bill number will match each other's payments and neither result means anything.
- **Never assume another test's data or settings.** Do not assume a customer, supplier, invoice or bill exists because a previous run created one, and do not assume the reserve floor, automatic limit or committed cash is what you set unless this run set it. Read the settings you depend on and assert on what you read.
- **Read balances before and after.** Take cash, suspense, inflow counts and incident counts before the action, then again after, and assert on the **difference**. An absolute figure is a claim about the whole workspace, including everyone else's activity; a delta is a claim about your action.
- **Count what you found before asserting on it.** A page scan or parser that matches zero rows and then asserts nothing about them passes over a broken parser.
- **Clean up by identity, not by wholesale reset.** Delete the records this run created, identified by their suffix. A reset that clears the workspace destroys another run's evidence — and `make testsprite-reset` is for the whole instance, not for tidying up one test.
- **Never write a credential into a tracked file.** The PIN lives in `.env`, which is gitignored. A test that hardcodes it, or a doc that quotes it, puts it in the repository history and makes it a secret that has to be rotated.
- **Two instances, one sandbox account.** Each instance only ingests the deposits it created and only acts on transfers whose request id it issued. Anything else is logged and ignored, never put in suspense or raised as an incident. A deposit you did not simulate may belong to the other instance.